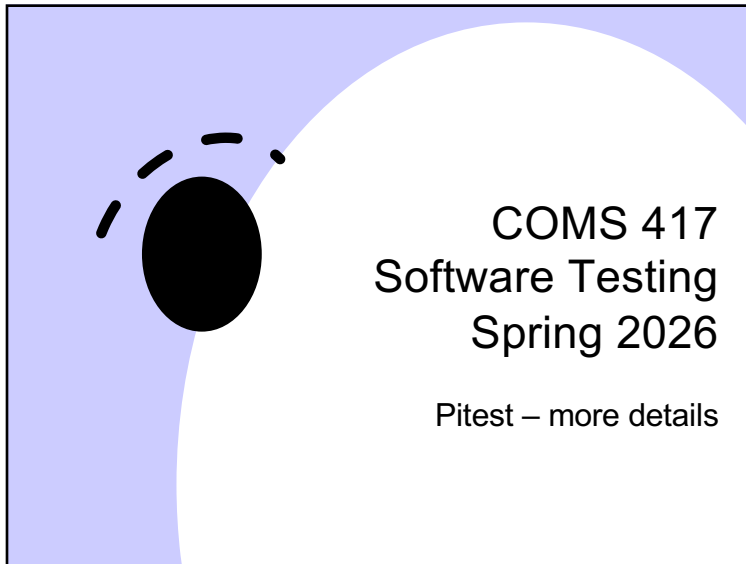
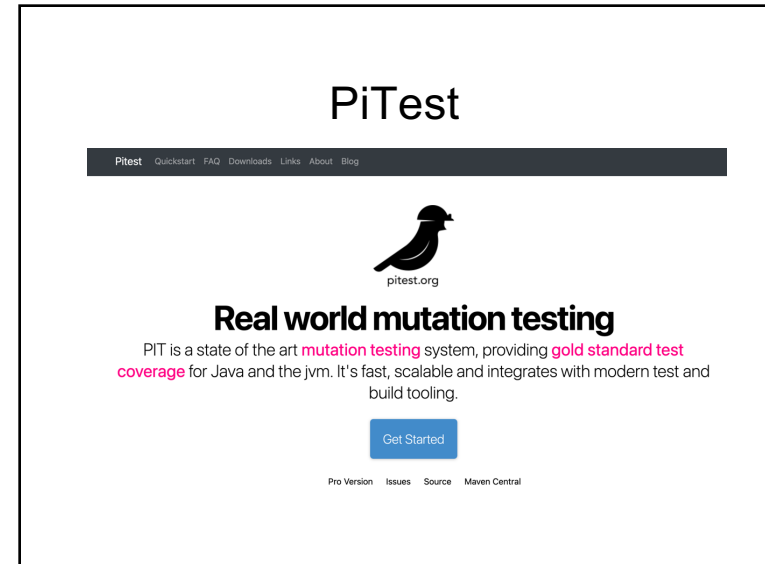
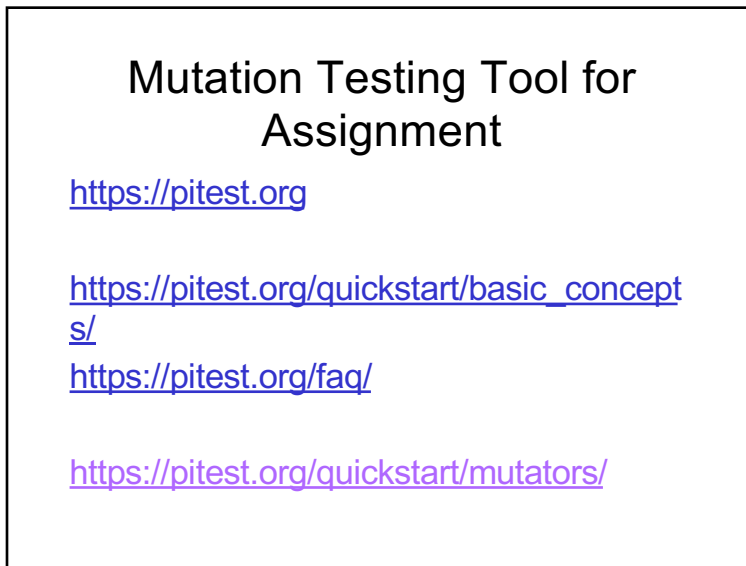




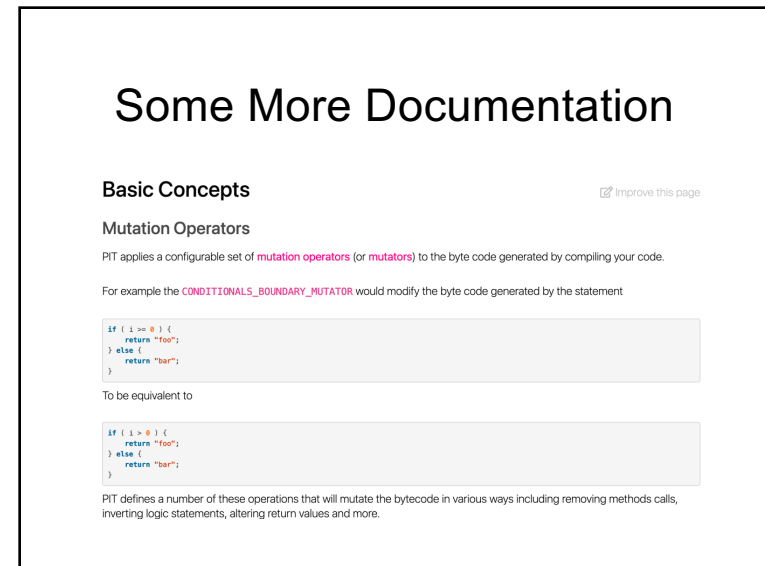
1



2



3



4

Some More Documentation

How does PIT choose which tests to run?

PIT chooses and prioritises tests based on three factors

- Line coverage
- Test execution speed
- Test naming convention

Per test case line coverage information is first gathered and all tests that do not exercise the mutated line of code are discarded. The remaining tests are then ordered by increasing execution time - test cases that belong to a class that is identified as a unit test for the mutated class are however weighted above other tests.

A class is considered to be the unit test for a particular class if it matches the standard JUnit naming convention of FooTest or TestFoo.

Unlike earlier systems PIT does **not** require that your tests follow this naming convention in order for it to work. Test names are used only as part of a heuristic to optimise run order.

5

Some More Documentation

How does PIT choose which tests to run?

PIT chooses and prioritises tests based on three factors

- Line coverage
- Test execution speed
- Test naming convention

Per test case line coverage information is first gathered and all tests that do not exercise the mutated line of code are discarded. The remaining tests are then ordered by increasing execution time - test cases that belong to a class that is identified as a unit test for the mutated class are however weighted above other tests.

A class is considered to be the unit test for a particular class if it matches the standard JUnit naming convention of FooTest or TestFoo.

Unlike earlier systems PIT does **not** require that your tests follow this naming convention in order for it to work. Test names are used only as part of a heuristic to optimise run order.

6

Getting Access to Mutated Code

Could I accidentally release mutated code if I use PIT?

No. The mutations that PIT generates are held in memory and never written to disk, except if explicitly enabled using the **EXPORT** feature. But even then they are only dumped inside the report directory and should not be released accidentally.

7

Operators

Overview

[Improve this page](#)

PIT currently provides some built-in mutators, of which most are activated by default. The default set can be overridden, and different operators selected, by passing the names of the required operators to the **mutators** parameter. To make configuration easier, some mutators are put together in groups. Passing the name of a group in the **mutators** parameter will activate all mutators of the group.

Mutations are performed on the byte code generated by the compiler rather than on the source files. This approach has the advantage of being generally much faster and easier to incorporate into a build, but it can sometimes be difficult to simply describe how the mutation operators map to equivalent changes to a Java source file.

The operators are largely designed to be **stable** (i.e not be too easy to detect) and minimise the number of equivalent mutations that they generate. Those operators that do not meet these requirements are not enabled by default.

Additional operators are available from [arcmutate](#).

8

Operators

Default Mutators

Conditionals Boundary Mutator (CONDITIONALS_BOUNDARY)

Active by default

The conditionals boundary mutator replaces the relational operators `<`, `<=`, `>`, `>=`

with their boundary counterpart as per the table below.

Original conditional	Mutated conditional
<code><</code>	<code><=</code>
<code><=</code>	<code><</code>
<code>></code>	<code>>=</code>
<code>>=</code>	<code>></code>

9

Operators

Increments Mutator (INCREMENTS)

Active by default

The increments mutator will mutate increments, decrements and assignment increments and decrements of local variables (stack variables). It will replace increments with decrements and vice versa.

For example

```
public int method(int i) {  
    i++;  
    return i;  
}
```

will be mutated to

```
public int method(int i) {  
    i--;  
    return i;  
}
```

Please note that the increments mutator will be applied to increments of **local variables only**. Increments and decrements of member variables will be covered by the [Math Mutator](#).

10

Math Mutator (MATH)

Active by default

The math mutator replaces binary arithmetic operations for either integer or floating-point arithmetic with another operation. The replacements will be selected according to the table below.

Original conditional	Mutated conditional
<code>+</code>	<code>-</code>
<code>-</code>	<code>+</code>
<code>*</code>	<code>/</code>
<code>/</code>	<code>*</code>
<code>%</code>	<code>*</code>
<code>&</code>	<code> </code>
<code> </code>	<code>&</code>
<code>^</code>	<code>&</code>
<code><<</code>	<code>>></code>
<code>>></code>	<code><<</code>
<code>>>></code>	<code><<<</code>
<code>-</code>	<code>.</code>

11